Appendix B

Annotated Answers to Review Questions

1 Pattern Matching for Java Objects

1.1 *(a), (c), and (d)*

Note that the traditional `switch` construct with `case` constants is used in this question

Option (b) fails to compile because the switch actions should be enclosed in a block when there is more than one statement in the switch action.

Option (e) fails to compile because it is implemented as a switch expression which must be exhaustive and that is not the case.

Option (f) fails to compile because it attempts to use both the colon (`:`) and the arrow (`->`) notations in the same switch block, which is not allowed.

1.2 *(a)*

Reference `x` is a static field that references a `Dog` object. The `instanceof` pattern match operator returns `true`. The pattern variable `y` shadows the static field `y` and is assigned the `Dog` object. Its scope is the `if` block. The `if` block is executed and prints `"dog"`. The pattern variable `y` goes out of scope. The `else` block is not executed. The print statement after the `if-else` construct refers to the static field `y` that references a `Cat` object and thus prints `"cat"`.

1.3 *(b)*

Options (a) (c) and (d) are valid, because the type `Placeholder<String>` of `p` is a sub-type of unbounded type `Placeholder<?>`, parameterized type `Placeholder<String>`, and raw type `Placeholder`. Thus casting it to a broader type is safe. However, the diamond operator `<>` is not allowed in the context of a generic pattern.

1.4 *(a) and (d)*

The `switch` expression requires pattern matching to be exhaustive, which could be achieved by explicitly listing the two permitted types of the `Animal` interface or by specifying the `default` label. That would be sufficient from the compiler point of view, however, considering that the parameter `animal` can be `null` and that the `default` label does not cover the `null` case, it needs to be added explicitly. It is also possible to create a `case` label that matches any `Animal` instead of the `default` label.

The parameter in this method is called `animal`, which means the same local variable `animal` cannot be used in a `case` label of a `switch` construct. Option (b) is thus not allowed.

The fact that the parameter is `null`, the switch expression will throw a `NullPointerException` at runtime for option (c) as it does not match the `null` value, and the `animal.toString()` method call in option (e) will do the same in option (e).

Options (a) and (d) make the `switch` expression exhaustive and match the `null` value.

1.5 *(b), (c), and (e)*

The `switch` expression must be exhaustive—that is, the switch labels must cover all values of the selector expression type. The case dominance rules ensure that `case` labels are reachable. Constant `case` labels and guarded pattern labels can be specified in any order, as long as they precede unguarded pattern labels. The `null` value must be matched by a `null` case label that must be specified before any `default` label which, if specified, must be the last switch label.

Option (a): The second `case` label, which is an unguarded `case` label, dominates the guarded `case` labels.

Option (b): Case dominance rules are satisfied.

Option (c): Case dominance rules are satisfied.

Option (d): The first case label, which is an unguarded `case` label, dominates the guarded `case` labels.

Option (e): Case dominance rules are satisfied.

1.6 *(b)*

The `switch` expression must be exhaustive—that is, the switch labels must cover all values of the selector expression type. The case dominance rules ensure that `case` labels are reachable. Constant `case` labels and guarded pattern labels can be specified in any order, as long as they precede unguarded pattern labels. The `null` value must be matched by a `null` case label that must be specified before any `default` label which, if specified, must be the last switch label. Only one match-all switch label is allowed in a switch block. The `null` case label and the `default` label can be combined.

Option (a): Incorrect syntax in the first `case` label. Case constants cannot be combined with the `||` operator.

Option (b): Case dominance rules are satisfied. The `null` case label and the `default` label are combined correctly.

Option (c): The `null case` label and the `default` label are not combined correctly.

Option (d): The `default` label cannot dominate other switch labels and must be the last switch label.

Option (e): Only one match-all switch label can be specified. Option (e) has two such labels: `Object o` case label and the `default` label.

1.7 *(a) and (d)*

Option (a) is correct. The `private` components of the `anOrder` record are accessed directly in the print statement, not via accessor methods. This is not an error as the record declarations are local inner records.

Option (b) is incorrect because local variables `name`, `price`, and `quantity` are also declared as pattern variables in the nested record pattern of the `instanceof` pattern match operator.

Option (c) is incorrect because local inner records are declared in the wrong order, record `Order` refers to record `Product` before it has been declared. It is also incorrect for the same reason as in option (b): duplication of local variables.

Option (d) is correct, because the order of the record field declarations is not relevant in this case, and although the pattern variables `name`, `price`, and `quantity` in the `instanceof` pattern match operator shadow the static field variables `name`, `price`, and `quantity`, they are allowed.

2 Sequenced Collections and Maps

2.1 *(d)*

A `SequencedSet` maintains the elements in insertion order and does not allow duplicates. The contents of the `values` sequenced set change as follows:

Click here to view code image

```
Add first 1:[1]
Add first 2:[2, 1]
Add last  3:[2, 1, 3]
Add last  2:[1, 3, 2] No duplication. Repositioning of 2 by the addLast() method.
Add       3:[1, 3, 2] No duplication. No repositioning by the add() method.
```

2.2 *(c) and (e)*

Option (a) throws an `UnsupportedOperationException` as `TreeSet` implements methods `addFirst()` and `addLast()` that always throw this exception. In this case, the `TreeSet` maintains its elements sorted according to the natural ordering of strings— that is, lexicographically. Adding an element first or last with the `addFirst()` or the `addLast()` method, respectively, can violate this natural ordering, and therefore explicit positioning of elements in a `Treeset` is not supported.

Options (b) and (d) result in a compile-time error as `HashSet` does not implement the `SequencedCollection` or the `SequencedSet` interface, respectively.

Option (c) is correct. Class `LinkedHashSet` implements the `SequencedSet` interface, which is a subtype of the `SequencedCollection` and it supports `addFirst()` and `addLast()` methods.

Option (e) is correct. In this case, the `add()` method will insert elements into the `TreeSet` sorted according to the natural ordering of strings.

2.3 *(a)*

Option (a): the `keys` set is `[C, B, A]`, as it is the reversed keyset of `map`. The map is reversed, but its result is discarded, so the `map` reference remains unchanged. The `values` collection of this map is still `[C, B, A]`. The first element of the `keys` set is equal to the first element of the `values` collection.

Option (b): the `keys` set is `[C, B, A]`, as it is the reversed keyset of `map`. The reference `map` is reassigned its reversed map, thus the order of its entries is `C=A`, `B=B`, `A=C`. The `values` collection of this map is `[A, B, C]`. The first element of the `keys` set is not equal to the first element of the `values` collection.

Option (c): the `keys` set is `[A, B, C]` and the `values` collection is `[C, B, A]`. Their first elements are not equal.

Option (d): the `keys` set is `[C, B, A]` and the `values` collection is `[A, B, C]`. Their first elements are not equal.

2.4 *(b)*

This code populates the linked hash map `lhm` with entries where the `Integer` value of `i` is the key and the `String` element from the `numbers` array with the index value of `i` is the value associated with the key.

The entries are inserted in `lhm` in the following order by the `for(;;)` loop as the value of `i` is incremented:

Click here to view code image

```
0=Zero  {0=Zero}
1=One   {1=One, 0=Zero}
2=Two   {1=One, 0=Zero, 2=Two}
2=Two   {2=Two, 1=One, 0=Zero}
3=Three {3=Three, 2=Two, 1=One, 0=Zero}
4=Four  {3=Three, 2=Two, 1=One, 0=Zero, 4=Four}
```

For the values of `i` not divisible by `2` (`i % 2 != 0`), the entry is put first in the map (i.e., for i equal to 1 and `3`), otherwise the entry is put last (i.e., for `i` equal to `0`, `2`, and `4`). However for `i` equal to `2`, the entry `(2, "Two")` is reinserted. The first time it is put last. The second time, since it already exists in the map, it is repositioned as the first entry of the map by the `putFirst()` method. Correct answer is (b).

3 Virtual Threads

3.1 *(d)*

The platform threads are non-daemon threads by default. This means that when the `main()` method terminates, any non-daemon threads that are still running will continue

to run until they complete. In the scenario above, the runnable object executes a loop that terminates only if the `break` statement is executed. The platform thread will continue to log the value of `i` until an `InterruptedException` is caught and the `break` statement is executed. It will log at least one value of `i` before the `InterruptedException` is caught and the platform thread completes execution.

3.2 *(b)*

The platform threads are non-daemon threads by default. This means that when the `main()` method terminates, any non-daemon threads that are still running will continue to run until they complete.

In the scenario above, the runnable object executes a loop which will only terminate if the `break` statement is executed. The condition in the `if` statement will never be `true`, because catching the `InterruptedException` clears the interrupted status of the thread—the `Thread.currentThread().isInterrupted()` call will always return `false`, so the loop will never execute the `break` statement. However, it will still log the value of `i` because the log statement in the `catch` block will always be executed when the exception is caught.

3.3 *(c) and (d)*

The virtual threads are daemon threads. This means that when the main thread (of the `main()` method) completes, any daemon threads that are still running will be terminated. It does not matter what code the virtual thread is running, it will be terminated once the main thread completes.

If the main thread completes before the virtual thread gets to run, nothing will be logged as the virtual thread will be terminated (option (c)).

If the virtual thread gets to run before the main thread completes, it will log at least one value of `i` before being terminated when the main thread completes (option (d)).

3.4 *(c) and (d)*

The virtual threads are daemon threads. This means that when the main thread (of the `main()` method) completes, any daemon threads that are still running will be terminated. It does not matter what code the virtual thread is running, it will be terminated once the main thread completes.

If the main thread completes before the virtual thread gets to run, nothing will be logged as the virtual thread will be terminated (option (c)).

If the virtual thread gets to run before the main thread completes, it might log one or more values of `i` before being terminated when the main thread completes (option (d)). In the scenario above, the runnable object executes a loop which will only terminate if the `break` statement is executed. The condition in the `if` statement will never be `true`, because catching the `InterruptedException` clears the interrupted status of the thread—the `Thread.currentThread().isInterrupted()` call will always return `false`, so the loop will never execute the `break` statement and the thread will continue to execute until it is terminated when the main thread completes.

3.5 *(a) and (f)*

Option (a): The priority of virtual threads cannot be changed. It is always `Thread.NORM_PRIORITY` (i.e., 5).

Option (b): JVM exits when all platform threads (non-daemon threads) that are running have completed their execution and any running virtual threads (daemon threads) have been terminated.

Option (c): Virtual threads do not change their priority when blocked on I/O operations, but rather they are unmounted on executing an I/O operation.

Option (d): Platform threads do not change their priority when blocked on I/O operations.

Option (e): Virtual threads are lightweight and do not benefit from the extra overhead of using a thread pool.

Option (f) Managing platform threads using a thread pool can improve performance as it minimizes the overhead due to thread creation—if a platform thread is available in the thread pool, it is used instead of creating a new one.

3.6 *(a), (c), and (d)*

Option (a): Unlike platform threads that are managed by the operating system, virtual threads are managed by the JVM.

Option (b): Virtual threads are not intended to be used for CPU-bound tasks, but rather for I/O-bound tasks.

Option (c): Existing concurrency codebases require minimal refactoring to use virtual threads. The enhanced concurrency API and the virtual thread management by the JVM allow seamless integration of virtual threads into existing code.

Option (d): A virtual thread is lightweight compared to a platform thread: less expensive to create and dispose of, requires less stack memory to manage its execution, and also context switching is less expensive. These factors increase the throughput of concurrency applications that use virtual threads.

Option (e): Virtual threads are designed for I/O-bound tasks, while platform threads are best suited for CPU-bound tasks, thus virtual threads are not intended to replace platform threads, but rather supplement them.

3.7 *(b) and (c)*

Option (a): A platform thread is called a carrier thread when a virtual thread is mounted and running on it.

Option (b) is correct. A carrier thread is a platform thread on which a virtual thread is mounted for execution.

Option (c) is correct. It has no name when created, but a name can be set with the `Thread.setName()` method. A virtual thread always has a unique thread ID.

Option (d): The JVM scheduler determines which platform thread a virtual thread is mounted on after it is ready to resume execution—there are no guarantees that it will be on the same platform thread.

3.8 *(h) and (i)*

Option (a): The static method `startVirtualThread()` of the `Thread` class creates and starts a new thread. Restarting it results in a `java.lang.IllegalThreadStateException`.

Option (b): The virtual thread is created and its name is set. The parent thread will wait for it to complete execution. However, if the virtual thread completes before its name is set in the main thread, the thread will have no name (although it will have a thread ID) and the task will print `NAME:`.

Option (c): Both `start()` and `unstarted()` methods of the virtual thread builder create a thread. Two virtual threads are created. The thread reference `vt` is reassigned to the second virtual thread with the name `vt_2`.

Option (d): There is no method `started()` of the virtual thread builder. The code will not compile.

Option (e): There is no method `name()` of the `Thread` class, but it has the `setName()` method for this purpose. The code will not compile.

Option (f): The `start()` method of the `Thread` class does not take a `Runnable` as an argument, but the `start()` method of the virtual thread builder does.

Option (g): The `start()` method of the `Thread` class returns `void`, not a `Thread`.

Option (h) is correct. The method chaining is correct, yielding a virtual thread. The `join()` method waits for the virtual thread to complete execution.

Option (i) is correct. The virtual thread that is created is not assigned to any reference.

3.9 *(a) and (b)*

Option (a) is correct. Although the thread factory can be customized to set the thread name, it can be overridden by the `setName()` method of the `Thread` class.

Option (b) is correct. The virtual thread factory is redundant. A platform thread is created that will print `NAME: vt_1`.

Option (c): The `newThread()` method of the thread factory creates a thread but does not start it, and the thread will not print anything.

Option (d): Cannot invoke the `join()` method on the `void` value returned by the `start()` method.

Option (e): The code will print `NAME: vt_2`. First, the virtual thread builder is used to create a thread with the name `vt_1`, but it is never started. The virtual thread factory then creates a thread with the name `vt_2`. This thread will result in `NAME: vt_2` being printed.

Updates